

Class XI | Unit 5 | weight 20%

Programming in C++

Board weight about 20% | 20 theory periods and 20 lab periods

This is the heaviest unit in Class XI. You will write complete C++ programs that calculate, decide, repeat, and call functions. Every program in this lecture was written to compile and run.

What you should be able to do

- Use C++ data types, operators, and cin and cout, including cascading.
- Write selection with if, if-else, else-if, switch and nested if.
- Write for, while, do-while and nested loops, and use break, continue and goto correctly.
- Declare, define and call functions. Explain scope, parameters, inline functions and exit.
- Pass constants, pass by value and pass by reference. Use default arguments and return values.
- Overload functions by the number and types of arguments.

Path through the lecture

- 1 Type the calculator on the first lab day. Do not stay on the board for the whole period.
- 2 Add decisions, then loops, then functions. Each lab program below is one of those steps.
- 3 Teach pass by reference with the before-and-after output on the screen.
- 4 End with overloading, and say clearly that return type alone does not overload a function.

5.1 Data, operators and streams

A C++ program for this course starts with the iostream header and a main function. cin reads from the keyboard. cout writes to the screen. The insertion operator << sends a value out. The extraction operator >> brings a value in. Cascading means chaining those operators in one statement.

Types you will use

Type	Holds	Example
int	Whole numbers	int marks = 76;
float	Numbers with a fractional part	float average = 72.5;
double	A fractional number kept more precisely	double tax = 0.05;
char	One character	char grade = 'A';
bool	true or false	bool pass = true;
void	No value, used for a function that only does work	void drawStars(int n);

Operators

Family	Operators	Meaning
Arithmetic	+ - * / %	Add, subtract, multiply, divide, remainder
Assignment	= += -=	Store a value, or change the stored value
Relational	== != > < >= <=	Compare two values. The result is true or false
Logical	&& !	And, or, not
Increment	++ --	Add 1 or subtract 1

Family	Operators	Meaning
Conditional	?:	Choose one of two values

Integer division drops the fraction: $7 / 2$ is 3. The remainder operator gives what is left: $7 \% 2$ is 1. Use `==` to compare and `=` to assign. Confusing those two is the most common logic error in a condition.

Program 5.1 Calculator, with cascading output

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;
    cout << "Sum = " << a + b << endl;
    cout << "Difference = " << a - b << endl;
    cout << "Product = " << a * b << endl;
    if (b != 0)
        cout << "Quotient = " << a / b << endl;
    else
        cout << "Cannot divide by zero" << endl;
    return 0;
}
```

The division line is guarded. Division by zero is undefined, so the program tests the divisor first. Sample run: input 12 3 prints sum 15, difference 9, product 36 and quotient 4.

5.2 Decisions

`if` runs a block only when the condition is true. `if-else` chooses one of two blocks. `else if` chains several conditions and the first true condition wins. A nested `if` is an `if` inside another `if`. `switch` compares one integer or character with several constant cases. End each case with `break`, otherwise execution falls into the next case. `default` runs when no case matches.

Program 5.2 A mark sheet using else-if

```
#include <iostream>
using namespace std;

int main() {
    int marks;
    cout << "Enter marks out of 100: ";
    cin >> marks;
    cout << "Grade: ";
    if (marks < 0 || marks > 100)
        cout << "Invalid" << endl;
    else if (marks >= 80)
        cout << "A" << endl;
    else if (marks >= 70)
        cout << "B" << endl;
    else if (marks >= 60)
        cout << "C" << endl;
    else if (marks >= 50)
        cout << "D" << endl;
    else
        cout << "Fail" << endl;
    return 0;
}
```

These grade bands are a classroom exercise so the selection is easy to test. They are not the board's own grading rules. Input 76 prints grade B. Input 110 prints Invalid because the mark is outside 0 to 100.

Program 5.3 Largest of three, using nested decisions

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cout << "Enter three integers: ";
    cin >> a >> b >> c;
    int largest = a;
    if (b > largest)
        largest = b;
    if (c > largest)
        largest = c;
    cout << "Largest = " << largest << endl;
    return 0;
}
```

Program 5.4 switch for the day of the week

```
#include <iostream>
using namespace std;

int main() {
    int day;
    cout << "Enter a day number (1 to 7): ";
    cin >> day;
    switch (day) {
        case 1: cout << "Monday" << endl; break;
        case 2: cout << "Tuesday" << endl; break;
        case 3: cout << "Wednesday" << endl; break;
        case 4: cout << "Thursday" << endl; break;
        case 5: cout << "Friday" << endl; break;
        case 6: cout << "Saturday" << endl; break;
        case 7: cout << "Sunday" << endl; break;
        default: cout << "Invalid day" << endl;
    }
    return 0;
}
```

5.3 Loops and jumps

A `for` loop is the natural choice when you know how many times to repeat. A `while` loop tests the condition before the body, so the body may never run. A `do-while` runs the body once and then tests, so it is the right loop for a menu. A nested loop has a loop inside a loop. The inner loop finishes its repetitions for each step of the outer loop.

Program 5.5 Prime or composite

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;
    if (n <= 1) {
        cout << n << " is neither prime nor composite" << endl;
        return 0;
    }
    bool isPrime = true;
    for (int i = 2; i <= n / 2; i++) {
        if (n % i == 0) {
            isPrime = false;
            break;
        }
    }
    if (isPrime)
        cout << n << " is prime" << endl;
    else
        cout << n << " is composite" << endl;
    return 0;
}
```

The number 1 is neither prime nor composite. A prime number is greater than 1 and has no factor other than 1 and itself. The loop tests divisors from 2 up to $n/2$. If any divisor divides exactly, the number is composite and break leaves the loop.

Program 5.6 Four digits on four lines

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a four-digit number: ";
    cin >> n;
    if (n < 1000 || n > 9999) {
        cout << "Enter a four-digit number" << endl;
        return 0;
    }
    cout << n / 1000 << endl;
    cout << (n / 100) % 10 << endl;
    cout << (n / 10) % 10 << endl;
    cout << n % 10 << endl;
    return 0;
}
```

Integer division by 1000 isolates the thousands digit. The remainder after division by 10 isolates the last digit. Input must be from 1000 to 9999.

Program 5.7 Factors of a number

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;
    cout << "Factors: ";
    for (int i = 1; i <= n; i++) {
        if (n % i == 0)
            cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

Program 5.8 A menu with do-while

```
#include <iostream>
using namespace std;

int main() {
    int choice, n;
    do {
        cout << "1. Square a number" << endl;
        cout << "0. Exit" << endl;
        cout << "Choice: ";
        cin >> choice;
        if (choice == 1) {
            cout << "Number: ";
            cin >> n;
            cout << "Square = " << n * n << endl;
        }
    } while (choice != 0);
    cout << "Done" << endl;
    return 0;
}
```

`break` leaves the nearest loop or switch immediately. `continue` skips the rest of this repetition and goes to the next test. `goto` jumps to a label. It is in the curriculum so you can recognise it. Do not use it in ordinary programs: a loop or a function states the intention more clearly, and a `goto` makes a long program hard to follow.

Program 5.9 `break`, `continue` and a labelled `goto`

```
#include <iostream>
using namespace std;

int main() {
    cout << "continue skips 3: ";
    for (int i = 1; i <= 5; i++) {
        if (i == 3)
            continue;
        cout << i << " ";
    }
    cout << endl;
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            cout << "Stopped at 3" << endl;
            break;
        }
    }
    int n = 1;
print_line:
    if (n == 1)
        cout << "goto reached the label" << endl;
    return 0;
}
```

5.4 Functions

A function is a named block of work. **Predefined** functions come with the language or its library, such as `sqrt` in `cmath`. **User-defined** functions are the ones you write. Functions let you name a task, test it once, and call it from several places. A large program becomes a set of small ones.

The **signature** is the function's name, its parameter list, and its return type. A **prototype** declares the function before `main`. The **definition** contains the body. A **call** uses the function. Formal parameters are the names in the definition. Actual parameters are the values sent at the call.

A **local variable** exists only inside the block where it is declared. A **global variable** is declared outside functions and can be used by all of them; use it rarely, because any function can change it. A **static local variable** keeps its value between calls to that function.

An **inline** function is a short function that the compiler may place at the point of the call, avoiding a call for a one-line calculation. `exit`, from `cstdlib`, ends the whole program at once. A `return` statement leaves only the current function and can carry a value back to the caller.

Program 5.10 A function that draws a triangle of stars

```
#include <iostream>
using namespace std;

void drawStars(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << "*";
        cout << endl;
    }
}

int main() {
    int size;
    cout << "Enter size: ";
    cin >> size;
    drawStars(size);
    return 0;
}
```

Program 5.11 Return the larger of two arguments

```
#include <iostream>
using namespace std;

int larger(int a, int b) {
    if (a > b)
        return a;
    return b;
}

int main() {
    int x, y;
    cout << "Enter two integers: ";
    cin >> x >> y;
    cout << "Larger = " << larger(x, y) << endl;
    return 0;
}
```

5.5 Passing arguments

- **By value:** the function receives a copy. Changing the parameter does not change the caller's variable.
- **By reference:** the parameter is an alias for the caller's variable, written with &. Changing it changes the original.
- A **const parameter** is a promise not to modify that parameter inside the function.
- A **default argument** is a value used when the caller omits that argument. Defaulted parameters go at the end of the list.

Program 5.12 Value, reference, const, inline and a default argument

```
#include <iostream>
using namespace std;

void byValue(int n) { n = 99; }

void byReference(int &n) { n = 99; }

void showConst(const int n) { cout << "Constant parameter: " << n << endl; }

inline int square(int n) { return n * n; }

int add(int a, int b = 10) { return a + b; }

int main() {
    int a = 10;
    byValue(a);
    cout << "After by value: " << a << endl;
    byReference(a);
    cout << "After by reference: " << a << endl;
    showConst(7);
    cout << "square(6) = " << square(6) << endl;
    cout << "add(5) = " << add(5) << endl;
    cout << "add(5, 3) = " << add(5, 3) << endl;
    return 0;
}
```

Read the output of Program 5.12 carefully. After the call by value the variable is still 10. After the call by reference it is 99. add(5) uses the default second argument and returns 15. add(5, 3) returns 8. square(6) returns 36.

Program 5.13 exit ends the program

```
#include <iostream>
#include <cstdlib>
using namespace std;

int visits = 0;

void leave() {
    cout << "Goodbye" << endl;
    exit(0);
}

int main() {
    int n;
    static int calls = 0;
    calls++;
    visits++;
    cout << "Enter a positive mark, or -1 to stop: ";
    cin >> n;
    if (n < 0)
        leave();
    cout << "Mark stored. calls = " << calls << endl;
    cout << "visits = " << visits << endl;
    return 0;
}
```

5.6 Overloading

Function overloading. Two or more functions with the same name and different parameter lists, in the same scope. The compiler chooses the function from the arguments at the call.

You may overload on the number of arguments or on the types of the arguments. You may not overload on the return type alone. Two functions called `sum` that both take two ints and differ only because one returns int and one returns double are rejected. The call would not tell the compiler which one you meant.

Program 5.14 Overloading on number and type of arguments

```
#include <iostream>
using namespace std;

int sum(int a, int b) { return a + b; }

int sum(int a, int b, int c) { return a + b + c; }

double sum(double a, double b) { return a + b; }

int main() {
    cout << "int sum = " << sum(3, 4) << endl;
    cout << "three sum = " << sum(3, 4, 5) << endl;
    cout << "double sum = " << sum(2.5, 3.0) << endl;
    return 0;
}
```

Exam point

In a board program, write the include line, using `namespace std;`, a complete main, and matching braces. Initialise variables before you read them in an expression. Test the program with the sample in the question and with one awkward value, such as 0 or 1.

Key terms

Term	Meaning to write in an answer
Operator	A symbol that asks for an operation on one or more values.

Term	Meaning to write in an answer
Cascading	Chaining << or >> in a single input or output statement.
Loop	A statement that repeats a block while a condition holds.
Prototype	A declaration of a function's name, return type and parameters.
Formal parameter	The parameter name written in the function definition.
Actual parameter	The value or variable supplied at the call.
Pass by reference	Passing a variable so the function can change the original.
Overloading	Same function name, different parameter lists.

Bring it together

- cin and cout, with << and >>, are the streams for this course.
- if, else-if and switch make decisions. for, while and do-while repeat. break and continue alter one loop.
- A function has a prototype, a definition and a call. It can return a value.
- Pass by value protects the original. Pass by reference changes it. Defaults fill in missing arguments.
- Overload by arguments, never by return type alone.

Do this in the practical

- 1 Build the calculator, the mark sheet, the prime test, the four-digit split and the factor list.
- 2 Write drawStars and larger as separate functions and call them from main.
- 3 Show, with printed output, that pass by value does not change the original and pass by reference does.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. Which operator finds the remainder?

- A. /
- B. %
- C. ==
- D. &&

2. A do-while loop:

- A. Never runs the body
- B. Tests after the body, so the body runs at least once
- C. Can only count down
- D. Replaces a function

3. break inside a loop:

- A. Skips to the next repetition
- B. Leaves the loop
- C. Deletes the file
- D. Overloads the function

4. A default argument must:

- A. Be the first parameter
- B. Appear at the end of the parameter list
- C. Be a pointer
- D. Be global

5. Which pair can be overloaded together?

- A. `int f(int)` and `double f(int)`
- B. `int f(int)` and `int f(int, int)`
- C. Two identical functions
- D. `int f(int)` and `int f(int)`

6. Pass by value:

- A. Sends a copy
- B. Always changes the caller's variable
- C. Needs the `&` symbol on the parameter
- D. Ends the program

7. The correct way to compare two ints for equality is:

- A. `a = b`
- B. `a == b`
- C. `a := b`
- D. `a eq b`

8. `exit(0)` :

- A. Leaves only the current loop
- B. Terminates the program
- C. Draws a star
- D. Opens a file

Answer key

Q	Answer	Why this is the answer
1	B	The <code>%</code> operator yields the remainder of integer division.
2	B	The test is at the bottom, so one execution happens before the test.
3	B	<code>break</code> terminates the nearest loop or switch.
4	B	Defaults are trailing parameters so the call stays unambiguous.
5	B	The parameter lists differ in the number of arguments. Return type alone is not enough.
6	A	The function works on a copy, so the original is unchanged.
7	B	<code>==</code> compares. A single <code>=</code> assigns.
8	B	<code>exit</code> ends the entire program. <code>return</code> leaves only the current function.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. What is cascading of the output stream? Write one statement.

Model answer

Cascading means using << more than once in a single statement. Example: `cout << "Sum = " << a + b << endl;` The text and the number are sent out by the same statement.

Q2. Differentiate while and do-while.

Model answer

while tests the condition before the body, so the body may not run at all. do-while runs the body and then tests, so the body runs at least once. do-while suits a menu that must appear before the user chooses.

Q3. Differentiate formal and actual parameters.

Model answer

Formal parameters are the names in the function heading, such as `int a` and `int b` in `int larger(int a, int b)`. Actual parameters are the values at the call, such as `larger(x, y)`. The actual values are passed into the formal parameters.

Q4. Why can two functions not be overloaded by return type alone?

Model answer

The compiler selects the overloaded function from the arguments at the call. If both functions take the same arguments, the call does not show which function is wanted, even if the return types differ. The program is rejected.

Q5. What does a static local variable do?

Model answer

A static local variable is created once and keeps its value after the function returns. The next call sees the value left by the previous call. An ordinary local variable is created again on each call.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain selection and iteration structures in C++, with the job of each.

- 1 Define selection as choosing a path, and name if, if-else, else-if, nested if and switch. Mention break and default.
- 2 Give a one-line situation for else-if, such as a grade, and for switch, such as a menu number.
- 3 Define iteration and compare for, while and do-while, including how many times the body can run.
- 4 Explain nested loops with the star triangle: the outer loop is the row, the inner loop prints the stars.
- 5 Name break and continue and give the effect of each.

Question 2. Explain user-defined functions, parameter passing and overloading.

- 1 Give the advantages: a named task, reuse, and a smaller main.
- 2 Show prototype, definition and call, and define the signature.
- 3 Contrast pass by value and pass by reference with the & sign and the effect on the original variable.
- 4 Explain default arguments and the return statement.
- 5 Define overloading and state the rule about argument lists and return types. Give sum of two ints and sum of three ints as the example.